

Database Project - Part 2

Group 10

Raj Maharjan (ist1115593) [33.33%, 22 hours]*
Ahmad Ashraf Zargar (ist1115594) [33.33%, 22 hours]*
Mohammad Alsalman (ist1115608) [33.33%, 22 hours]*

Laboratory Teacher: Prof. Paulo Carreira, Prof. Francisco Regateiro

January 2026

* All group members contributed equally to this work.

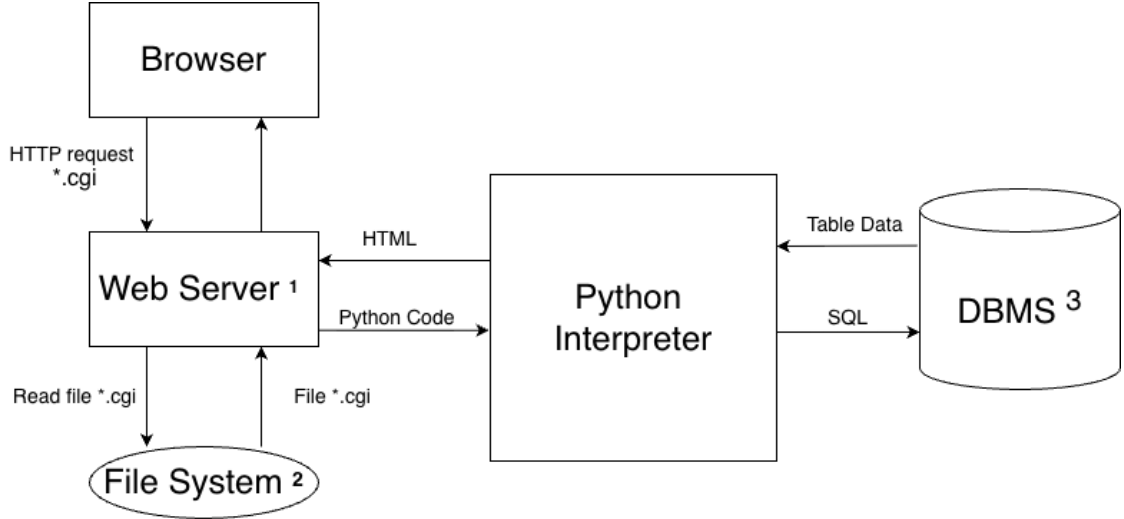
1 Integrity Constraints

The project implements two key integrity constraints using database triggers and stored procedures to enforce important business rules. The first constraint (IC-1) ensures that every sailor in the system must be classified as either a Junior or a Senior sailor; no sailor can exist without one of these designations. This was implemented using a deferred constraint trigger that checks whenever a new sailor is added to the database. The reason we made this trigger deferred is because when inserting a new sailor, the system would normally check immediately that the sailor exists in either the Junior or Senior table, but at that moment the sailor hasn't been added to either table yet. Since we also cannot add a Junior or Senior sailor first due to foreign key constraints (they must reference an existing sailor), making the constraint deferred allows us to insert the sailor and then add them to the appropriate table before the final check occurs. Additionally, we created separate triggers on the Junior and Senior tables to ensure that a sailor cannot be both Junior and Senior at the same time, maintaining the disjoint specialization rule.

The second constraint (IC-2) prevents trip dates from overlapping within the same reservation. This trigger checks whenever a trip is inserted or updated to ensure that its take-off and arrival dates do not conflict with any other trip for the same boat and reservation. The implementation carefully handles the overlap detection by comparing the new trip's dates with existing trips, and when updating an existing trip, it excludes the trip being updated from the comparison to avoid false conflicts with itself. Together, these constraints work automatically in the background to maintain data integrity and prevent invalid scenarios from occurring in the database.

2 System Architecture

When the browser requests a page (an HTTP request to a *.cgi endpoint), the web server running on the Técnico host (istxxxxxx@sigma.tecnico.ulisboa.pt) receives the request and loads the corresponding *.cgi script from the server's file system (the /web directory). The web server then hands the script to the Python interpreter, which executes the Python code and, whenever needed, sends SQL queries to the DBMS (hosted on Sigma). The DBMS returns the requested table data back to the Python interpreter, which formats the results into HTML. Finally, that generated HTML is returned to the web server and sent back as the HTTP response to the browser, completing the request/response cycle as shown in Figure 1.



1. Located at istxxxxxx@sigma.tecnico.ulisboa.pt
2. The file system is istxxxxxx@sigma.tecnico.ulisboa.pt/web
3. The database is stored in the tecnico server and accessed via DataGrip

Figure 1: Web Application Architecture Block Diagram

3 Files Description (Role of Each File)

Users typically enter from `index.cgi`, then go to `sailors.cgi` to create/remove sailors (which sends form data to `save_sailor.cgi` or a link to `remove_sailor.cgi`), and those sailor emails then become selectable inputs in `reservations.cgi` as the responsible senior and the authorized crew. When a reservation is created, `reservations.cgi` posts the reservation details to `save_reservation.cgi`, which writes the reservation and its initial authorisations to the database; afterwards, the same reservation can be opened in `authorise.cgi` by passing its composite key as URL parameters (`start_date`, `end_date`, `country`, `cni`) so the script knows exactly which reservation's crew to add/remove. Trips follow the same pattern: `trips.cgi` reads existing reservations from the database and encodes the selected reservation into a single "reservation key" (`start|end|country|cni`) that gets submitted to `save_trip.cgi`, which decodes it and inserts the trip under the correct reservation; deleting trips similarly passes the trip identifier (takeoff + reservation/boat key) to `remove_trip.cgi`. The following figure 2 displays the hierarchical file structure.

To prevent SQL Injection, all database interactions use parameterized SQL queries with `psycopg2` named parameters, instead of concatenating raw user input into SQL strings. This approach ensures user input is treated as data, not executable SQL. In our codebase, all inserts/deletes/queries that use user-provided values follow this parameter binding style.

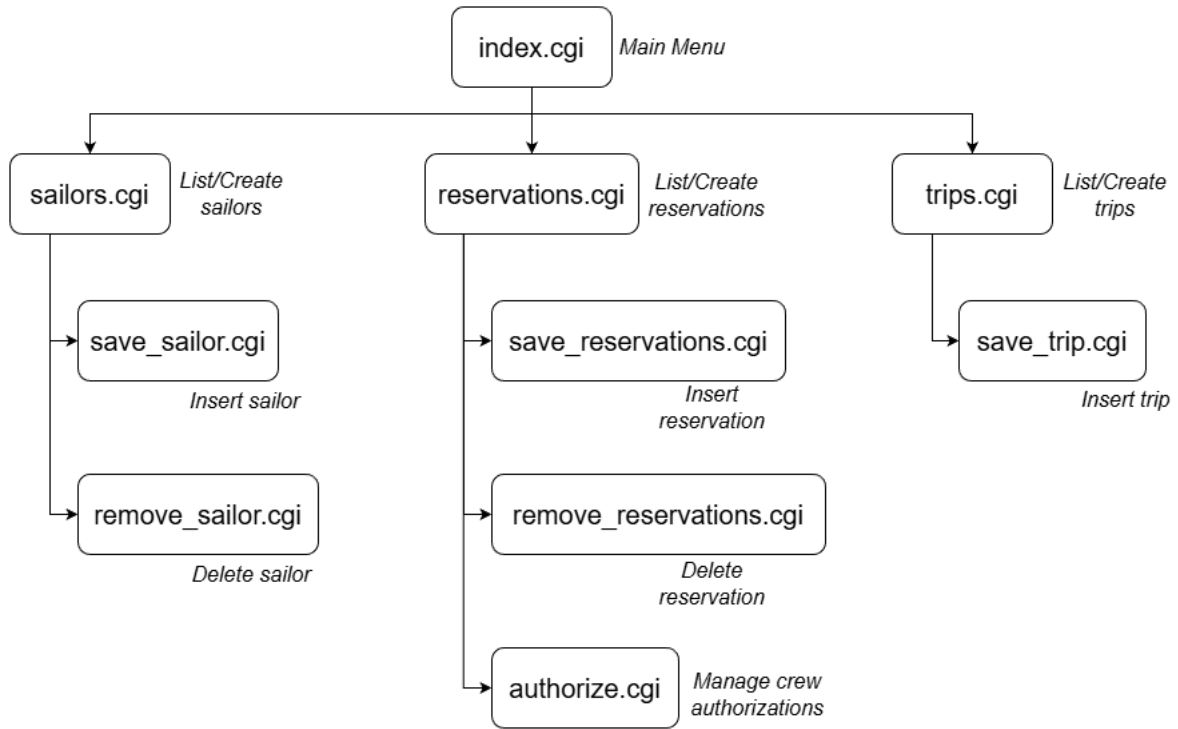


Figure 2: Hierarchical File Structure

3.1 index.cgi

This is the entry point of the webpage and acts as a simple landing page. It provides a visual menu linking the user to the three functional areas of the system: managing sailors, managing reservations, and managing trips. No database modifications happen here, its purpose is navigation and a clean starting point for web application.

3.2 Sailors Module

3.2.1 sailors.cgi

This page is the main “Sailors” dashboard. It renders the form used to register a new sailor (first name, surname, email, and disjoint mandatory specialization), and it also lists all existing sailors currently stored in the database. Each row includes a remove button that redirects to **remove_sailor.cgi** with the sailor’s email as the identifier.

Because it only reads and displays data (and links to actions), it does not directly trigger integrity constraints.

3.2.2 save_sailor.cgi

This script is called when the user submits the “Register New Sailor” form. It inserts the sailor into **sailor** and then inserts the same email into exactly one specialization

table (`junior` or `senior`), depending on the chosen rank. Both inserts happen inside a single transaction as to enforce the disjoint and mandatory specialization integrity constraint. This ensures that every sailor belongs to either Junior or Senior.

3.2.3 `remove_sailor.cgi`

This script deletes a sailor from one of the specialization table, (`junior` or `senior`) and the `sailor` record within the encapsulated transaction. If the sailor is referenced elsewhere (for example, as a reservation responsible senior or as a skipper in trips), the database rejects the deletion due to foreign key constraints; the script catches the error, rolls back, and displays a message indicating that dependent records must be removed first.

3.3 Reservations Module

3.3.1 `reservations.cgi`

This page is the main “Reservations” dashboard. It renders the reservation creation form, the “Responsible Senior” is filled by selecting sailors from the `senior` specialization, and the “Authorized Sailors” list is loaded from the full `sailor` list. After the form, it prints a table of existing reservations, where we can perform two actions on each reservation: either displaying in more detail or deleting the reservation.

3.3.2 `save_reservation.cgi`

This script creates a reservation from the submitted form using the composite primary key (`start_date`, `end_date`, `country`, `cni`). After inserting the reservation, the script automatically inserts authorization rows into `authorised` for the selected sailors.

All these steps run inside one transaction, which would be rolled back if an error occurs.

3.3.3 `authorise.cgi`

This page manages the “authorized crew” of a specific reservation. It receives the reservation composite primary key (start date, end date, boat country, and CNI), displays the reservation context, then lists all sailors currently authorized for it. The same page also allows you to authorize new sailors or revoke authorization (Note: Responsible sailor cannot be deauthorized).

3.3.4 `remove_reservation.cgi`

This script deletes a reservation and performs that process safely inside a single transaction: it first deletes the reservation’s authorizations, then deletes the trip logs linked to that reservation, and only then deletes the reservation itself. If any step fails, the transaction is rolled back.

3.4 Trips Module

3.4.1 `trips.cgi`

This page is the main “Trips” dashboard. It renders the form used to log a trip: locations are loaded from the `location` table, and reservation options are loaded from the `reservation` table. The lower half of the page prints the trip logbook (existing trips ordered by takeoff) and provides a delete link per trip that redirects to `remove_trip.cgi`.

3.4.2 `save_trip.cgi`

This script inserts a new trip. It parses the reservation key selected by the user, parses the origin/destination coordinates, and then runs one `INSERT` into the `trip` table within a transaction. This is the key file where the application directly interacts with the trip-related integrity rules enforced in the database:

- The schema CHECK constraints enforce date validity (`takeoff ≥ reservation_start_date`, `takeoff ≤ arrival`, and `arrival ≤ reservation_end_date`).
- The trigger `tg_check_trip_overlap` ensures that trips under the same reservation do not overlap in time. If the user attempts to create an overlapping trip, the trigger raises an exception; the script catches it, rolls back the transaction, and displays an error message instead of inserting invalid data.
- The “skipper must be authorized” rule is enforced by the database logic. When this constraint fails, the script shows a message indicating the skipper is not authorized i.e not a part of the reservation.

3.4.3 `remove_trip.cgi`

This script deletes a trip using its composite primary key (takeoff + reservation identifiers + boat identifiers). The deletion is performed inside a transaction. If the key is valid, the record is removed and the user is returned to the trips page with a success message; otherwise, the deletion fails safely with rollback and an error display.

3.5 templates.py

This file centralizes the HTML layout used by every CGI page so the application has a consistent look and structure without repeating code. It contains two main functions that work together to frame all the content that gets displayed to users. The `print_header()` function prints out the beginning of the webpage, including the title and all the styling instructions that make the page look good. We used Bootstrap, which is a ready-made collection of design tools, to help build the page quickly with pre-designed components. We also used CSS, which is the language for controlling colors, fonts, spacing, and overall appearance, to customize those components with our own nautical theme and styling. The `print_footer()` function prints out the ending of the webpage, closing all the necessary tags and including the code libraries needed to make interactive features work.

3.6 login.py

The `login.py` file stores all the important connection information needed to access the database where the project's data is kept.

4 Data Flow

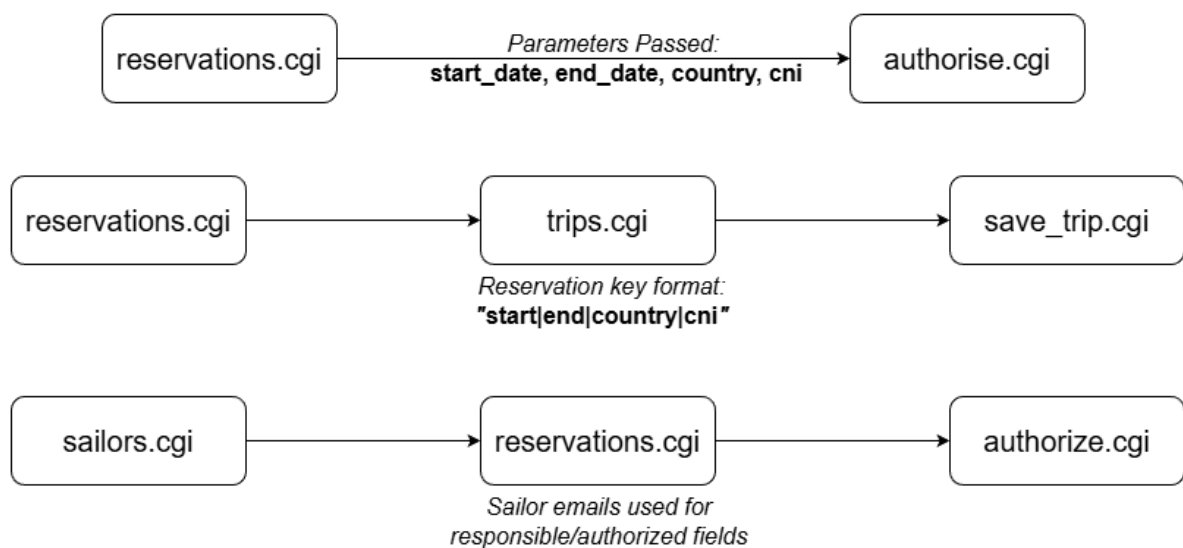


Figure 3: Data Flow Relationships

In Figure 3, in the first interaction, `reservations.cgi` passes the reservation details, namely the **start date**, **end date**, **country**, and **CNI** to `authorize.cgi`. This information uniquely identifies the reservation and provides the necessary context to authorize sailors for that specific reservation. Using these parameters, `authorize.cgi` can asso-

ciate authorized sailors with the correct reservation and ensure that authorizations are applied to the intended time period and location.

In the second interaction, `reservations.cgi` passes a reservation key to `trips.cgi`, formatted as `start|end|country|cni`. Since each trip must be associated with an existing reservation, this key allows `trips.cgi` to retrieve and display only the trips related to the selected reservation. When a new trip is created or booked, `trips.cgi` forwards the same reservation information to `save_trip.cgi`, ensuring that the trip is correctly stored in the database and linked to the appropriate reservation.

In the third interaction, `sailors.cgi` provides sailor-related information, specifically **sailor email addresses**, to `reservations.cgi`. These emails are used to assign sailors to a reservation, for example to authorize sailors or to designate a senior sailor as responsible for the reservation. This information is then passed to `authorize.cgi`, where the authorization status of each sailor is recorded and associated with the corresponding reservation.